

Database development with PL/SQL INSY 8311



ADVENTIST UNIVERSITY
OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | eric.maniraguha@auca.ac.rw | [LinkedIn Profile](#)



6h00 pm – 8h50 pm

- Monday A -G209
- Tuesday B-G209
- Wednesday E-G209
- Thursday F-G308

April 2025

Database development with PL/SQL

Reference reading

- [13 Data Concurrency and Consistency](#)
- [11 Data Concurrency and Consistency](#)



Lecture 12 - Transactions and Concurrency

Transactions: Fundamental Concepts

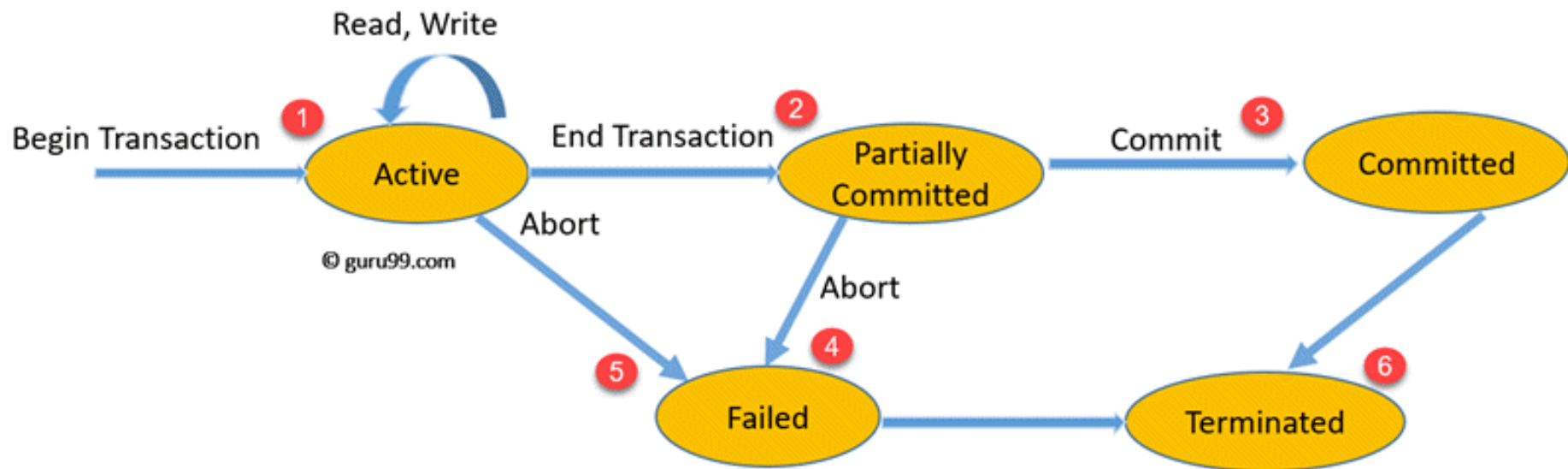
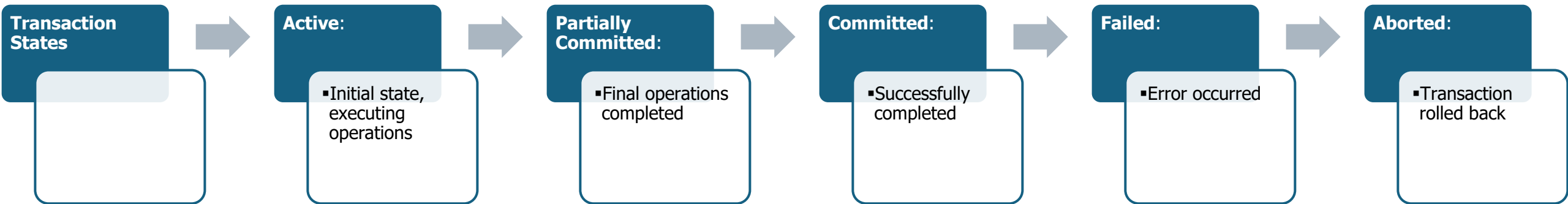
Definition & Example

- A transaction is a sequence of database operations treated as a single logical unit of work
- Ensures data integrity and consistency
- **Examples:** Transferring money between bank accounts, placing an order online, updating employee data.

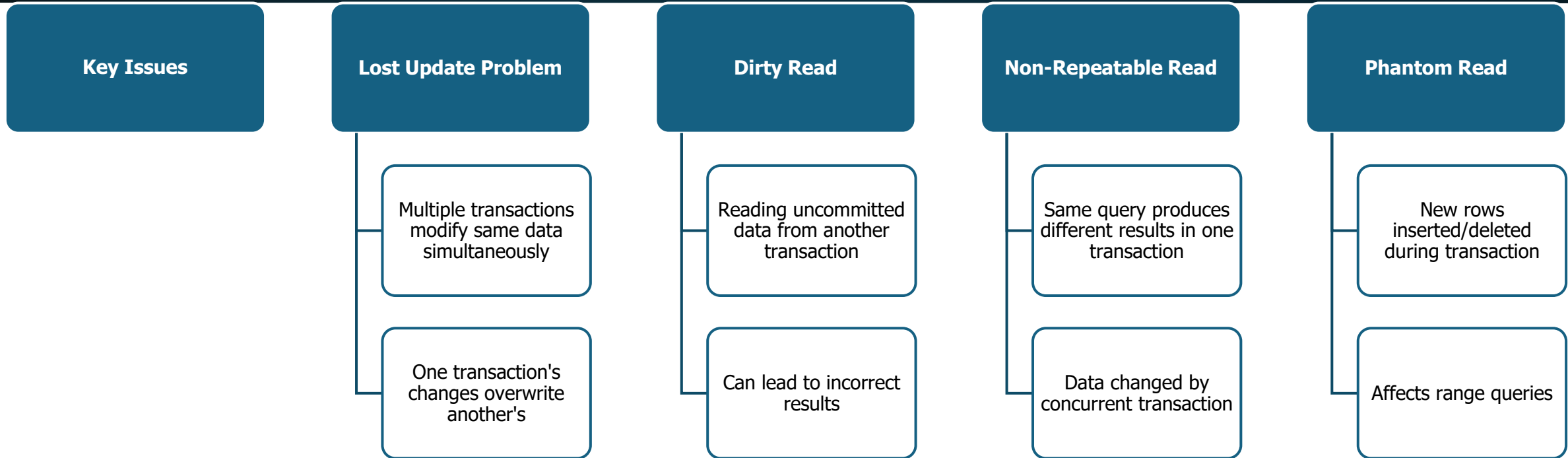
Follows ACID properties:

- **Atomicity:** All operations complete or none complete
- **Consistency:** Database remains in a valid state before and after transaction
- **Isolation:** Transactions do not interfere with each other
- **Durability:** Completed transactions are permanently saved

Transaction States



Concurrency Control Problems



Concurrency Control Techniques & Isolation Levels

Concurrency Control Techniques



☐ Lock-Based Mechanisms

- ☐ **Shared Locks (Read Locks)**
- Multiple transactions can read data simultaneously
 - Prevents write operations

- ☐ **Exclusive Locks (Write Locks)**
- Only one transaction can modify data
 - Blocks other read/write operations

Isolation Levels



☐ Read Uncommitted

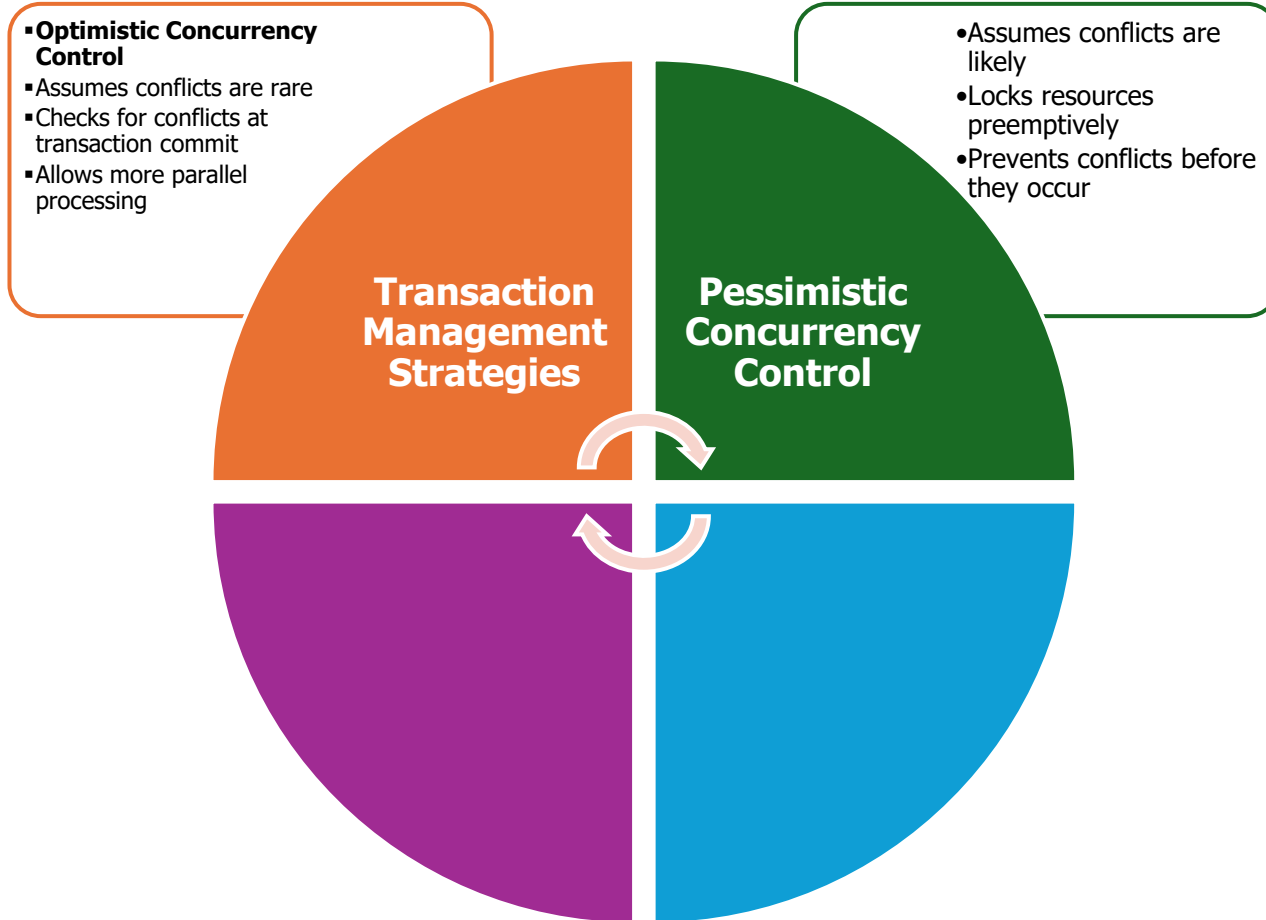
- Lowest isolation
- Allows dirty reads
- Maximum concurrency, minimum consistency

- ☐ **Read Committed**
- Prevents dirty reads
 - Default in many database systems

- ☐ **Repeatable Read**
- Prevents dirty and non-repeatable reads
 - May allow phantom reads

- ☐ **Serializable**
- Highest isolation level
 - Completely prevents concurrent anomalies
 - Lowest performance, highest consistency

Transactions and Concurrency in Databases



Implementation Considerations



Choose appropriate isolation level

Balance between consistency and performance

Use transaction boundaries carefully

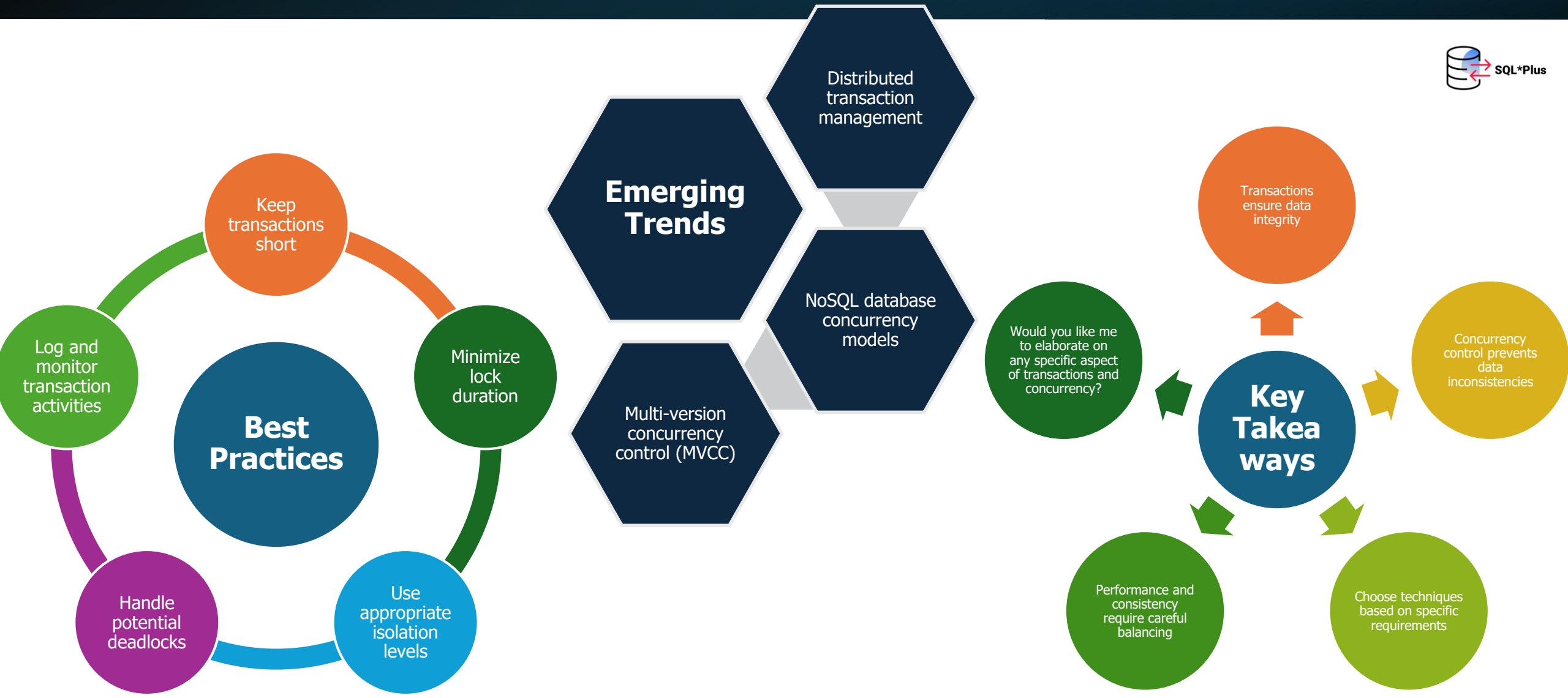
Implement proper error handling

Consider database-specific concurrency features

Code Example (Pseudocode)

```
BEGIN TRANSACTION;  
-- Check account balance  
UPDATE accounts SET balance = balance - 100 WHERE id = 123;  
UPDATE accounts SET balance = balance + 100 WHERE id = 456;  
COMMIT;
```

Best Practices & Emerging Trends & Key Takeaways



Beyond Basics: Advanced PL/SQL Challenges for the Curious Mind

YouTube Tutorial

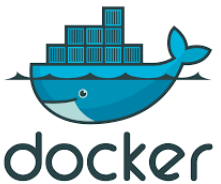
1. [SQL Data Warehouse from Scratch | Full Hands-On Data Engineering Project](#)
2. [Complete a project with me! - Building Data Model and Database](#)
3. [Football Data Analytics | Azure End To End Data Engineering Project](#)
4. [Airlines Data Ingestion | End-To-End Data Engineering Project | AWS Services | COMPLETE TUTORIAL](#)
5. [End-to-End DWBI Project: Real-Life Data Integration with Snowflake, Python, SQL & PowerBI](#)



ArcGIS Pro

Articles

- [Roadmap for Data Engineering 2023](#)
- [How To Master The Data Engineer Roadmap For Career Advancement](#)
- [Roadmap for Data Engineering 2025](#)
- [30+ Data Engineering Projects for Beginners in 2025](#)



Note: Clicking on any of these images will direct you to a tutorial that can help you learn more about the technology.

Beyond Basics: Advanced PL/SQL Challenges for the Curious Mind - Open-Source Apps

The Modern Open-source Data Stack

 Datafold

Collection + Integration

 rudderstack  JITSU  Airbyte  SNOWFLOW 

Warehousing

 trino
(Formerly PrestoSQL)   pinot  druid  ClickHouse

Transformation

 dbt   PREFECT  beam  Flink  DAGSTER

Data Cataloging

 Amundsen 

Analysis

 Metabase  Lightdash  Superset  jupyter  PostHog
 redash  Plausible  Querybook

Quality Assurance

 great_expectations

Closing Remarks

As we bring this PL/SQL course to a close, I sincerely hope that you now possess a solid foundation in both the **theoretical principles** and **practical skills** of PL/SQL programming. Throughout this journey, you have learned how to:

- Declare and manipulate **variables** effectively.
- Apply **control structures** to manage the flow of logic.
- Create and use **procedures** and **functions** for modular code.
- Handle **exceptions** to build robust and fault-tolerant programs.
- Write **efficient**, **reliable**, and **maintainable** PL/SQL applications.



Key Takeaways

- **Practice** – Keep coding. Build projects, solve real problems, and reinforce your understanding through hands-on experience.
- **Explore** – Don't stop here. Go deeper into advanced topics like **performance tuning**, **security**, and **PL/SQL optimization**.
- **Collaborate** – Learn with others. Join communities, ask questions, share knowledge, and grow together.

You are now equipped with the tools and mindset to **succeed in your database career**. The road ahead will be filled with opportunities and challenges—but remember, each line of code, each debug session, and each project brings you closer to mastery.

Biblical Message & Inspirational Quote

Biblical Message

"For I know the plans I have for you," declares the Lord, "plans to prosper you and not to harm you, plans to give you a hope and a future."

— **Jeremiah 29:11**

This powerful verse reminds us that beyond the technical journey lies a deeper purpose—one filled with hope, growth, and success. Even when challenges arise, know that your path is being shaped for greatness.

Inspirational Quote

"Success is the sum of small efforts, repeated day in and day out."

— **Robert Collier**

Your progress in PL/SQL, like any worthwhile endeavor, is built through **consistency**, **persistence**, and **daily effort**. Stay committed. Stay curious.

Congratulations on reaching this milestone. Keep pushing your limits and never stop learning. I'm excited to see where your journey takes you next—and I'm confident it will be somewhere amazing.

Go forth and code with purpose.